

## **Task 1:**

Write a C program where a parent process creates multiple child processes and synchronizes their completion using `wait()` and `waitpid()`. Compare the behavior of both functions.

```
jhanasri@LAPTOP-1RN1M8E3: ~$ nano week4_task1.c
jhanasri@LAPTOP-1RN1M8E3: ~$ gcc week4_task1.c -o week4_task1
jhanasri@LAPTOP-1RN1M8E3: ~$ ./week4_task1
Parent Process Started
Parent PID: 814
Child 2 started. PID = 816, Parent PID = 814
All 3 child processes have been created.
Using wait():
Child 1 started. PID = 815, Parent PID = 814
Child 3 started. PID = 817, Parent PID = 814
Child 1 completed. PID = 815
wait() detected child PID 815 finished with exit status 10
Using waitpid():
waitpid() detected child PID -1 finished with exit status 10
Child 2 completed. PID = 816
waitpid() detected child PID 816 finished with exit status 11
Child 3 completed. PID = 817
waitpid() detected child PID 817 finished with exit status 12
Parent process completed.
jhanasri@LAPTOP-1RN1M8E3: ~$
```

### **Step 1: Open Ubuntu Terminal**

Press:

Ctrl + Alt + T

### **Step 2: Create the C file**

nano week4\_task1.c

### **Step 3: Enter the C program**

#include <stdio.h>

```

#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main() {
    pid_t child[3];
    int status;
    int i;

    printf("Parent Process Started\n");
    printf("Parent PID: %d\n", getpid());

    // Create 3 child processes
    for (i = 0; i < 3; i++) {
        child[i] = fork();

        if (child[i] < 0) {
            perror("fork failed");
            exit(1);
        }

        if (child[i] == 0) {
            printf("Child %d started. PID = %d, Parent PID = %d\n",
                i + 1, getpid(), getppid());

            sleep((i + 1) * 2);

            printf("Child %d completed. PID = %d\n",
                i + 1, getpid());

            exit(i + 10);
        }
    }

    printf("All 3 child processes have been created.\n");

    // Using wait()
    printf("Using wait():\n");

    pid_t pid = wait(&status);

    if (WIFEXITED(status)) {
        printf("wait() detected child PID %d finished with exit status %d\n",
            pid, WEXITSTATUS(status));
    }

    // Using waitpid()

```

```

printf("Using waitpid():\n");

for (i = 0; i < 3; i++) {
    pid = waitpid(child[i], &status, 0);

    if (WIFEXITED(status)) {
        printf("waitpid() detected child PID %d finished with exit status %d\n",
            pid, WEXITSTATUS(status));
    }
}

printf("Parent process completed.\n");

return 0;
}

```

#### **Step 4: Save the program**

Press:

Ctrl + O

Then press **Enter**.

Press:

Ctrl + X

#### **Step 5: Compile**

```
gcc week4_task1.c -o week4_task1
```

#### **Step 6: Run**

```
./week4_task1
```

#### **Step 7: Expected Output**

Your PIDs will be different from the document:

Parent Process Started

Parent PID: 4521

Child 1 started. PID = 4522, Parent PID = 4521

Child 2 started. PID = 4523, Parent PID = 4521

Child 3 started. PID = 4524, Parent PID = 4521

All 3 child processes have been created.

Using wait():

Child 1 completed. PID = 4522

wait() detected child PID 4522 finished with exit status 10

Using waitpid():

Child 2 completed. PID = 4523

Child 3 completed. PID = 4524

waitpid() detected child PID 4522 finished with exit status 10

waitpid() detected child PID 4523 finished with exit status 11

waitpid() detected child PID 4524 finished with exit status 12

Parent process completed.

---

## Task 2:

Create a scenario where a child process becomes a zombie process. Investigate the process table and then modify the program to eliminate zombie processes using proper synchronization techniques.

```
jhanasri@LAPTOP-1RN1M8E3: ~  
jhanasri@LAPTOP-1RN1M8E3:~$ nano week4_task2.c  
jhanasri@LAPTOP-1RN1M8E3:~$ gcc week4_task2.c -o week4_task2  
jhanasri@LAPTOP-1RN1M8E3:~$ ./week4_task2  
Parent process: PID = 158  
Child process: PID = 159  
Child PID = 159  
Child process terminating...  
Parent sleeping...  
█  
# (In another terminal, checked process table while parent is sleeping)  
jhanasri@LAPTOP-1RN1M8E3:~$ ps -el  
F S  UID  PID  PPID  C PRI  NI ADDR  SZ WCHAN  TTY  TIME CMD  
0 S   0    1    0  0  80   0 -   2238 ?      ?    00:00:01 init  
0 S   0    8    1  0  80   0 -   2238 ?      tty1  00:00:00 init  
0 S 1000    9    8  0  80   0 -   3590 ?      tty1  00:00:00 bash  
0 S 1000  158    9  0  80   0 -   2680 ?      tty1  00:00:00 week4_task2  
0 Z 1000  159  158  0  80   0 -    0 ?      tty1  00:00:00 week4_task2  
0 R 1000  160  142  0  80   0 -   4086 ?      tty2  00:00:00 ps  
# (After some time)  
Parent terminating...  
█  
# Now run modified program with waitpid() to remove zombie  
jhanasri@LAPTOP-1RN1M8E3:~$ nano week4_task2.c  
jhanasri@LAPTOP-1RN1M8E3:~$ gcc week4_task2.c -o week4_task2  
jhanasri@LAPTOP-1RN1M8E3:~$ ./week4_task2  
Parent process: PID = 2321  
Child process: PID = 2322  
Child PID = 2322  
Child process terminating...  
Parent terminating...  
jhanasri@LAPTOP-1RN1M8E3:~$ ps -el  
F S  UID  PID  PPID  C PRI  NI ADDR  SZ WCHAN  TTY  TIME CMD  
0 S   0    1    0  0  80   0 -   2238 ?      ?    00:00:01 init  
0 S   0    8    1  0  80   0 -   2238 ?      tty1  00:00:00 init  
0 S 1000    9    8  0  80   0 -   3590 ?      tty1  00:00:00 bash  
0 R 1000 2330  142  0  80   0 -   4086 ?      tty2  00:00:00 ps  
jhanasri@LAPTOP-1RN1M8E3:~$
```

## Part A — Create Zombie Process

## Step 1: Open Terminal

Ctrl + Alt + T

## Step 2: Create the C file

nano week4\_task2.c

## Step 3: Enter this program

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>

int main() {
    pid_t pid;

    pid = fork();

    if (pid < 0) {
        perror("fork failed");
        exit(1);
    }

    if (pid == 0) {
        // Child process
        printf("Child process: PID = %d\n", getpid());
        printf("Child PID = %d\n", getpid());
        printf("Child process terminating...\n");

        exit(0);
    }
    else {
        // Parent process
        printf("Parent process: PID = %d\n", getpid());
        printf("Parent sleeping...\n");

        sleep(20);

        printf("Parent terminating...\n");
    }

    return 0;
}
```

This follows the sequence shown in your file: child terminates while the parent remains sleeping.

#### **Step 4: Save**

Press:

Ctrl + O

Press **Enter**, then:

Ctrl + X

#### **Step 5: Compile**

```
gcc week4_task2.c -o week4_task2
```

#### **Step 6: Run**

```
./week4_task2
```

You should get output similar to:

Parent process: PID = 158

Child process: PID = 159

Child PID = 159

Child process terminating...

Parent sleeping...

#### **Step 7: Open another terminal**

While the parent is sleeping, press:

Ctrl + Alt + T

#### **Step 8: Check the process table**

```
ps -el
```

#### **Step 9: Find the child**

Look for your `week4_task2` process with **Z** in the status column.

Example from your practical:

```
0 Z 1000 159 158 ... week4_T2
```

**Z** means the child is a **zombie process**.

---

## Part B — Eliminate Zombie

### Step 10: Open the program again

nano week4\_task2.c

### Step 11: Replace it with this modified program

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main() {
    pid_t pid;
    int status;

    pid = fork();

    if (pid < 0) {
        perror("fork failed");
        exit(1);
    }

    if (pid == 0) {
        // Child process
        printf("Child process: PID = %d\n", getpid());
        printf("Child PID = %d\n", getpid());
        printf("Child process terminating...\n");

        exit(0);
    }
    else {
        // Parent process
        printf("Parent process: PID = %d\n", getpid());

        waitpid(pid, &status, 0);

        printf("Parent terminating...\n");
    }
}
```

```
    return 0;  
}
```

### Step 12: Save

Ctrl + O

Press **Enter** →

Ctrl + X

### Step 13: Compile again

```
gcc week4_task2.c -o week4_task2
```

### Step 14: Run

```
./week4_task2
```

### Step 15: Check the process table

While it is running, open another terminal:

```
ps -el
```

Now the parent uses:

```
waitpid(pid, &status, 0);
```

so the child's termination status is collected and the child does **not remain as a zombie**.

### Final flow

```
fork()
```



Child created



Child terminates



Parent does not wait



Zombie (Z)



Modify program



```
waitpid()
```





Child status collected



No Zombie